ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

		Скобцов Ю.А.
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 6

Оптимизация путей на графах с помощью муравьиных алгоритмов
Вариант 4
по курсу: Эволюционные методы проектирования программно-информационных систем

РАБОТУ ВЫПОЛНИЛ
СТУДЕНТ

гр. №	4332		Могилатов С.И.
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ 4

1.1 Индивидуальное задание 4

1.2 Математическая модель 4

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 5

2.1 Построение маршрута 5

2.2 Испарение и обновление феромона 5

2.3 Случайный поиск и критерий остановки 5

3 АЛГОРИТМ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ 6

3.1 Последовательность работы программы 6

3.2 Параметры эксперимента 6

4 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА 7

4.1 Гамильтонов путь 7

4.2 Цикл коммивояжёра 7

4.3 Влияние параметров колонии 9

4.4 Сопоставление с лабораторной работой № 3 10

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 11

ЗАКЛЮЧЕНИЕ 12

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 13

ПРИЛОЖЕНИЕ А 14

Модуль муравьиной колонии 14

ПРИЛОЖЕНИЕ Б 16

Сценарий экспериментов 16

ВВЕДЕНИЕ

Муравьиные алгоритмы применяют для дискретных задач, где решение можно представить последовательностью переходов по графу. В этой работе исследуются два связанных случая на наборе Bayg29: открытый гамильтонов путь, проходящий через каждый город ровно один раз, и замкнутый тур задачи коммивояжёра.

Цель работы — реализовать муравьиный алгоритм для варианта 4, графически показать найденные маршруты и ход сходимости, сопоставить длину тура с эталонным решением, а также оценить влияние числа муравьёв и скорости испарения феромона.

Работа включает постановку задачи, краткое описание алгоритма, воспроизводимые параметры экспериментов, таблицы и графики результатов, сравнение с контрольным туром и ответы на контрольные вопросы. Требования к составу и оформлению отчёта взяты из методических указаний и опубликованных правил ГУАП [1; 2].

1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ

1.1 Индивидуальное задание

Для варианта 4 используется набор Bayg29 — 29 городов Баварии. В первой части требуется найти гамильтонов путь и показать его графически и по этапам поиска. Во второй части требуется решить задачу коммивояжёра, построить цикл и сравнить его с эталонным туром. Методические указания также предлагают сопоставить результат с лабораторной работой № 3 и исследовать влияние вероятностей кроссовера и мутации [1].

Кроссовер и мутация являются операторами генетических алгоритмов и в реализованном муравьином алгоритме не используются. Поэтому экспериментальная часть варьирует параметры, непосредственно управляющие колонией: число муравьёв и коэффициент испарения. В репозитории лабораторная работа № 3 посвящена задаче 0/1-рюкзака, а не маршрутизации Bayg29; длины решений этих разных задач численно несопоставимы. Сравнение с ЛР3 поэтому проводится на уровне принципов поиска, а качество тура — по эталонному циклу Bayg29.

1.2 Математическая модель

Каждый город — вершина полного взвешенного графа. Вес ребра между городами i и j равен евклидову расстоянию $d_{ij} = \sqrt{((x_i - x_j)^2 + (y_i - y_j)^2)}$, рассчитанному по координатам из приложения к методическим указаниям. Маршрут является перестановкой номеров 1–29.

Для открытого пути длина $L_п = \Sigma\, d(v_k, v_{k+1})$, $k=1…28$. Для цикла добавляется ребро возврата: $L_ц = L_п + d(v_{29}, v_1)$. Эталонная последовательность Bayg29 задаёт цикл длиной 9074,148. После удаления замыкающего ребра её длина равна 8791,836; это допустимый контрольный открытый путь, но не доказанный оптимум задачи о пути.

Итак, сравниваются две постановки на одной и той же матрице расстояний: путь заканчивается в последнем посещённом городе, а тур включает ещё одно ребро между последним и первым городами.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Построение маршрута

Муравьиная колония строит множество пробных решений. Каждый муравей начинает в случайном городе и последовательно выбирает ещё не посещённый город, пока не сформирует полную перестановку. Вероятность перехода зависит от накопленного феромона τ_{ij} и эвристической привлекательности короткого ребра $\eta_{ij} = 1/d_{ij}$:

$p_{ij} = \tau_{ij}^{\alpha}\ \eta_{ij}^{\beta} / \Sigma_{l \in N}\ \tau_{il}^{\alpha}\ \eta_{il}^{\beta}$, где N — множество непосещённых городов. Параметр α задаёт влияние опыта предыдущих муравьёв, а β — предпочтение коротких рёбер. В программе $\alpha=1$ и $\beta=3$.

2.2 Испарение и обновление феромона

После построения маршрутов феромон испаряется: $\tau_{ij} \leftarrow (1-\rho)\tau_{ij}$. Затем каждое ребро маршрута получает добавку $\Delta\tau = Q/L$, где Q — константа, а L — длина найденного маршрута. В реализации обновление симметрично для неориентированных рёбер. Испарение не даёт старым решениям полностью подавить исследование других маршрутов; добавка Q/L усиливает рёбра, входящие в более короткие маршруты.

2.3 Случайный поиск и критерий остановки

Алгоритм повторяет построение решений заданное число итераций и сохраняет лучшее найденное решение. После каждой итерации записываются длина лучшего маршрута к текущему моменту и средняя длина маршрутов колонии. Эти данные позволяют просматривать поиск по итерациям в подробном режиме и на графике сходимости.

3 АЛГОРИТМ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Последовательность работы программы

1. Загрузить координаты 29 городов Bayg29 и вычислить симметричную матрицу евклидовых расстояний.
2. Инициализировать небольшое одинаковое количество феромона на каждом ребре и создать колонию.
3. Для каждого муравья выбрать случайный начальный город и построить маршрут, каждый город посещая один раз.
4. Рассчитать длину пути или цикла, обновить глобальное лучшее решение и записать показатели итерации.
5. Испарить феромон, добавить Q/L на ребра каждого маршрута и перейти к следующей итерации.
6. Сохранить результаты, изображения маршрутов, график сходимости и CSV параметрического исследования.

После каждой итерации сохраняются лучшее и среднее значения длины, а допустимость итогового маршрута проверяется по числу вершин и отсутствию повторов. Исходный код алгоритмического модуля приведён в приложении А, сценарий экспериментов — в приложении Б.

3.2 Параметры эксперимента

Базовые серии используют фиксированные начальные значения генератора случайных чисел для повторяемости. Путь и цикл запускаются отдельно; основная серия содержит 40 муравьёв и 200 итераций. Параметрическое исследование выполняет по 120 итераций для каждой комбинации. Настройки двух серий сведены в таблицу 2.

Таблица 2 — Настройки алгоритма

Параметр	Базовый запуск	Исследование
Число муравьёв	40	20 или 40
Итерации	200	120
α — влияние феромона	1,0	1,0
β — влияние расстояния	3,0	3,0
Испарение ρ	0,35	0,25 или 0,45
Константа Q	100	100
Начальные значения	seed 44 (путь), 45 (цикл)	seed 51–54

4 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА

4.1 Гамильтонов путь

В первой серии найден открытый путь длиной 8403,049. Маршрут проходит через все 29 городов по одному разу и не содержит возвратного ребра. Для ориентира он на 4.42% короче пути длиной 8791,836, полученного удалением замыкающего ребра из эталонного тура. Такое сравнение показывает качество найденного решения относительно контрольного маршрута, но не доказывает оптимальность открытого пути.

Последовательность посещения: 3 → 29 → 26 → 5 → 9 → 12 → 6 → 28 → 1 → 21 → 2 → 20 → 10 → 4 → 15 → 19 → 16 → 13 → 24 → 27 → 8 → 23 → 7 → 25 → 11 → 22 → 14 → 18 → 17. На рисунке 1 показана геометрия найденного пути.

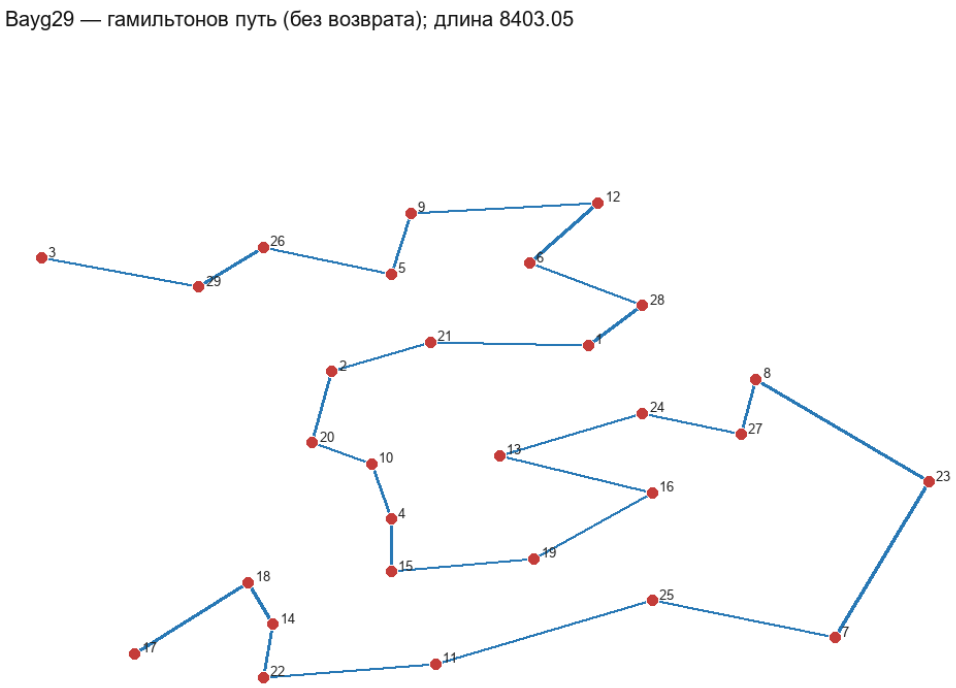


Рисунок 1 — Найденный гамильтонов путь Bayg29

4.2 Цикл коммивояжёра

Лучший найденный замкнутый тур имеет длину 9328,340 при эталонном значении 9074,148. Отклонение составляет +2.80% относительно

контрольного тура. Маршрут проходит через каждый город один раз и возвращается в начальный город: 5 → 21 → 2 → 20 → 10 → 4 → 15 → 18 → 14 → 22 → 17 → 11 → 19 → 25 → 7 → 23 → 27 → 16 → 13 → 24 → 8 → 1 → 28 → 6 → 12 → 9 → 26 → 29 → 3 → 5.

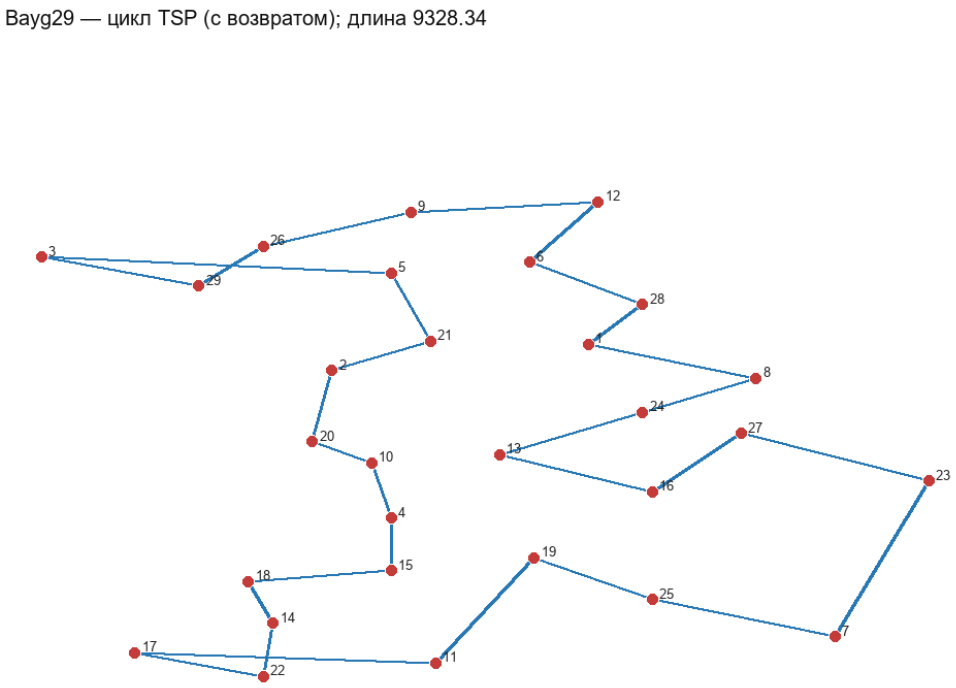


Рисунок 2 — Найденный замкнутый тур Bayg29

На рисунке 3 показано изменение лучшей длины по итерациям. Ступенчатые участки означают, что в этой части поиска глобально лучшее решение не изменилось; снижение кривой фиксирует появление более короткого маршрута.

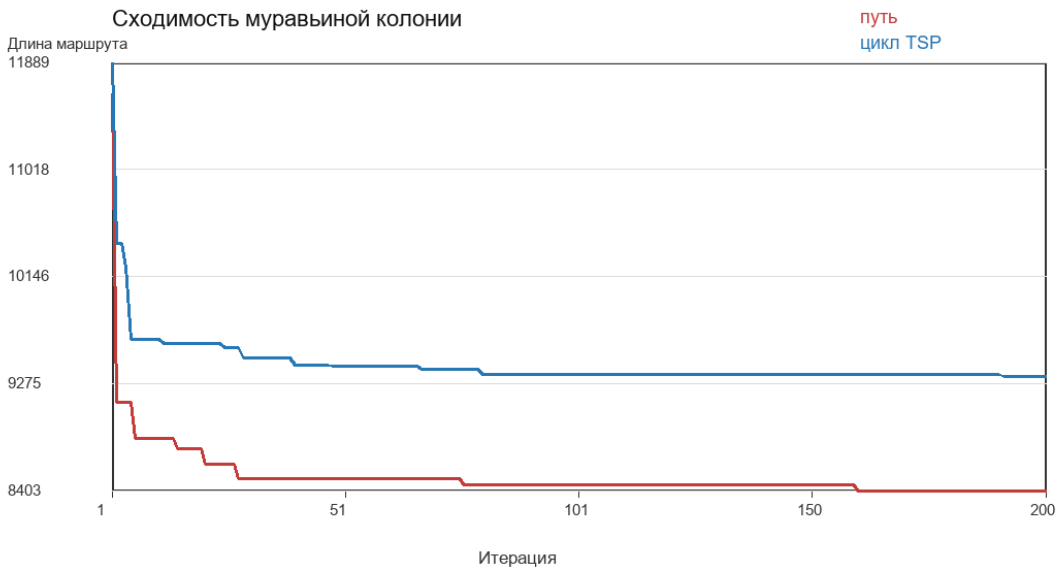


Рисунок 3 — Сходимость алгоритма для пути и цикла

Сводные численные показатели обеих серий приведены в таблице 3.

Таблица 3 — Итоги базовых серий

Задача	Найденная длина	Контрольная длина	Разница, %	Время, с
Открытый путь	8403.049	8791.836 (допустимый путь)	-4.42%	0.978
Замкнутый цикл	9328.340	9074.148 (эталон)	+2.80%	0.967

4.3 Влияние параметров колонии

В эксперименте изменялись число муравьёв и коэффициент испарения при одинаковых α, β, Q и числе итераций. Таблица 4 показывает лучшую длину цикла и измеренное время каждой серии. Это исследование параметров муравьиного метода заменяет запрошенную в формулировке методички вариацию кроссовера и мутации, которых в данном алгоритме нет.

Таблица 4 — Результаты исследования параметров

Муравьи	Испарение ρ	Лучшая длина	К эталону	Время, с
20	0.25	9391.118	+3.49%	0.289
40	0.25	9298.129	+2.47%	0.585
20	0.45	9421.491	+3.83%	0.305
40	0.45	9298.129	+2.47%	0.608

В этой серии 40 муравьёв дали более короткие туры, чем 20 муравьёв, но почти удвоили время расчёта. При 40 муравьях обе проверенные скорости испарения привели к одинаковой лучшей длине; при 20 муравьях значение

ρ=0,25 оказалось немного лучше ρ=0,45. Четыре запуска недостаточны для общего вывода о превосходстве одной настройки: стохастический поиск зависит от seed, а измеренное время — от нагрузки компьютера.

4.4 Сопоставление с лабораторной работой № 3

В имеющемся отчёте ЛР3 решается задача 0/1-рюкзака генетическим алгоритмом. У неё иной тип решения и другая целевая функция, поэтому сравнивать численные значения целевой функции с длинами туров Bayg29 нельзя. На уровне метода обе работы используют популяционный стохастический поиск и повторное улучшение решений. Генетический алгоритм оперирует хромосомами и генетическими операторами; муравьиный алгоритм строит маршруты и передаёт опыт через феромон. Для количественной оценки ЛР6 использован эталонный цикл Bayg29 из методички.

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Потенциальное решение представляется допустимым маршрутом — последовательностью вершин графа.

2. Феромон τ_{ij} отражает накопленный опыт использования ребра; его величина возрастает после прохождения по нему муравьёв.

3. Испарение уменьшает вклад старых маршрутов и помогает исследовать альтернативы; на ребре τ обновляется умножением на $(1-\rho)$.

4. В простом муравьином алгоритме агенты строят пути вероятностно, оценивают их длину, испаряют феромон и усиливают рёбра хороших решений.

5. α управляет влиянием феромона, β — эвристики короткого ребра $1/d_{ij}$; большие значения сильнее смещают выбор к соответствующей информации.

6. Качество маршрута для данной задачи оценивается суммарной длиной его рёбер; меньшее значение означает лучшее решение.

7. Критериями остановки могут быть лимит итераций, приемлемое значение качества или отсутствие улучшения в течение заданного числа итераций.

8. Основные параметры: число муравьёв, α, β, скорость испарения ρ, константа феромона Q, число итераций и начальный seed.

9. Общий цикл: инициализация, построение маршрутов, вычисление качества, испарение и добавление феромона, сохранение лучшего результата, проверка останова.

10. С генетическими алгоритмами муравьиные алгоритмы объединяют популяция решений, случайный поиск и итерационное усиление перспективных вариантов.

11. Генетический алгоритм изменяет хромосомы отбором, кроссовером и мутацией; муравьиный алгоритм строит решения последовательно, используя эвристику и распределённый феромонный след.

ЗАКЛЮЧЕНИЕ

Реализован муравьиный алгоритм для варианта 4 на данных Bayg29. Программа строит допустимый гамильтонов путь и отдельный замкнутый цикл, сохраняет маршруты, графики сходимости, историю эксперимента и сводную таблицу.

Для открытого пути получена длина 8403.049; для цикла — 9328.340, что на 2.80% выше эталонного значения. Исследование параметров показало компромисс между качеством и временем: увеличение колонии с 20 до 40 муравьёв улучшило найденный тур в проверенных сериях, но увеличило расчётное время.

Пошаговую динамику качества можно просмотреть в подробном режиме программы, а её графическое представление — на кривых сходимости. Эталонный тур является контрольной точкой для TSP; удаление его замыкающего ребра используется только как допустимый ориентир для задачи открытого пути. Следующий шаг для более надёжных выводов — повторить параметрические серии на нескольких seed и сравнить распределения длин маршрутов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Методические указания. Лабораторная работа № 6 «Оптимизация путей на графах с помощью муравьиных алгоритмов». 30 с. (Предоставленный учебный материал).

2. Санкт-Петербургский государственный университет аэрокосмического приборостроения. Для учебного процесса: правила оформления учебных работ и титульные листы лабораторных работ. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 05.11.2026).

3. Лабораторная работа № 3. Решение задачи 0/1-рюкзака генетическим алгоритмом: отчёт, использованный для качественного сопоставления методов. Репозиторий other-unc-tasks, ветка предмета.

ПРИЛОЖЕНИЕ А

Модуль муравьиной колонии

```
"""Ant colony optimization for open Hamiltonian paths and TSP cycles."""


from __future__ import annotations


import math
import random
import time
from dataclasses import dataclass



def validate_route(route: list[int], city_count: int) -> bool:
    return len(route) == city_count and set(route) == set(range(city_count))



def route_length(route: list[int], distances: list[list[float]], closed: bool) -> float:
    total = sum(distances[a][b] for a, b in zip(route, route[1:]))
    if closed and len(route) > 1:
        total += distances[route[-1]][route[0]]
    return total



@dataclass
class ColonyResult:
    route: list[int]
    length: float
    closed: bool
    history: list[dict]
    elapsed_seconds: float



class AntColony:
    def __init__(self, distances: list[list[float]], ants: int = 40, iterations: int = 300,
                 alpha: float = 1.0, beta: float = 3.0, evaporation: float = 0.35,
                 q: float = 100.0, seed: int = 4):
        self.distances = distances
        self.n = len(distances)
        if self.n < 2 or any(len(row) != self.n for row in distances):
            raise ValueError("distance matrix must be square and contain at least two cities")
        if not 0 < evaporation < 1:
            raise ValueError("evaporation must be between 0 and 1")
        self.ants = ants
        self.iterations = iterations
        self.alpha = alpha
        self.beta = beta
        self.evaporation = evaporation
        self.q = q
        self.rng = random.Random(seed)


    def _construct(self, pheromone: list[list[float]], closed: bool) -> list[int]:
        route = [self.rng.randrange(self.n)]
        remaining = set(range(self.n)) - set(route)
        while remaining:
            current = route[-1]
            candidates = sorted(remaining)
            weights = []
            for city in candidates:
                distance = max(self.distances[current][city], 1e-12)
                weights.append((pheromone[current][city] ** self.alpha) * ((1.0 / distance) ** self.beta))
            total = sum(weights)
            if total <= 0 or not math.isfinite(total):
                next_city = self.rng.choice(candidates)
            else:
```

```
                threshold = self.rng.random() * total
                next_city = candidates[-1]
                for city, weight in zip(candidates, weights):
                    threshold -= weight
                    if threshold <= 0:
                        next_city = city
                        break
            route.append(next_city)
            remaining.remove(next_city)
        return route


    def run(self, closed: bool = True) -> ColonyResult:
        started = time.perf_counter()
        initial_pheromone = 1.0 / (self.n * max(sum(min(row[j] for j in range(self.n) if j != i) for i,
row in enumerate(self.distances)) / self.n, 1e-9))
        pheromone = [[initial_pheromone if i != j else 0.0 for j in range(self.n)] for i in range(self.n)]
        best_route: list[int] | None = None
        best_length = math.inf
        history = []
        for iteration in range(1, self.iterations + 1):
            routes = []
            for _ in range(self.ants):
                route = self._construct(pheromone, closed)
                length = route_length(route, self.distances, closed)
                routes.append((route, length))
                if length < best_length:
                    best_route, best_length = route, length
            for i in range(self.n):
                for j in range(self.n):
                    pheromone[i][j] *= (1.0 - self.evaporation)
            for route, length in routes:
                deposit = self.q / max(length, 1e-12)
                edges = list(zip(route, route[1:]))
                if closed:
                    edges.append((route[-1], route[0]))
                for a, b in edges:
                    pheromone[a][b] += deposit
                    pheromone[b][a] += deposit
            history.append({"iteration": iteration, "best_length": best_length,
                            "iteration_best": min(length for _, length in routes),
                            "mean_length": sum(length for _, length in routes) / len(routes)})
        assert best_route is not None
        return ColonyResult(best_route, best_length, closed, history, time.perf_counter() - started)
```

ПРИЛОЖЕНИЕ Б

Сценарий экспериментов

```
"""Run Bayg29 ant-colony experiments for path search and the TSP."""


from __future__ import annotations


import argparse
import csv
import json
import math
from pathlib import Path


from PIL import Image, ImageDraw, ImageFont


from ant_colony import AntColony, route_length, validate_route



ROOT = Path(__file__).resolve().parent
RESULTS = ROOT / "results"
RESULTS.mkdir(exist_ok=True)
FONT = r"C:\Windows\Fonts\arial.ttf"


COORDINATES = [
    (1150,1760),(630,1660),(40,2090),(750,1100),(750,2030),(1030,2070),(1650,650),
    (1490,1630),(790,2260),(710,1310),(840,550),(1170,2300),(970,1340),(510,700),
    (750,900),(1280,1200),(230,590),(460,860),(1040,950),(590,1390),(830,1770),
    (490,500),(1840,1240),(1260,1500),(1280,790),(490,2130),(1460,1420),(1260,1910),(360,1980),
]
OPTIMAL_TOUR_1BASED = [1,28,6,12,9,26,3,29,5,21,2,20,10,4,15,18,14,17,22,11,19,25,7,23,8,27,16,13,24]



def distance_matrix(coordinates=COORDINATES):
    return [[math.dist(a, b) for b in coordinates] for a in coordinates]



def format_run(label: str, result, reference: float | None = None) -> str:
    route = [city + 1 for city in result.route]
    text = (f"{label}: длина {'цикла' if result.closed else 'открытого пути'}={result.length:.3f}; "
            f"время={result.elapsed_seconds:.3f} с; маршрут: " + " -> ".join(map(str, route)))
    if result.closed:
        text += f" -> {route[0]}"
    if reference is not None:
        difference = (result.length-reference)/reference*100
        text += f"; контрольное значение={reference:.3f}; разница={difference:+.2f}%"
    return text



def selected_history(history):
    rows = [row for row in history if row["iteration"] == 1 or row["iteration"] % 25 == 0]
    if history and rows[-1]["iteration"] != history[-1]["iteration"]:
        rows.append(history[-1])
    return rows



def draw_route(result, path: Path, title: str):
    canvas = Image.new("RGB", (1100, 760), "white")
    draw = ImageDraw.Draw(canvas)
    font = ImageFont.truetype(FONT, 20)
    xs = [point[0] for point in COORDINATES]; ys = [point[1] for point in COORDINATES]
    margin = 70
    def pos(index):
        x, y = COORDINATES[index]
```

```
        return (margin + (x-min(xs))/(max(xs)-min(xs))*(1000-2*margin),
                670 - (y-min(ys))/(max(ys)-min(ys))*(600-2*margin))
    edges = list(zip(result.route, result.route[1:]))
    if result.closed:
        edges.append((result.route[-1],result.route[0]))
    for a,b in edges:
        draw.line((*pos(a),*pos(b)), fill="#2878b5", width=3)
    for i in range(len(COORDINATES)):
        x,y=pos(i); draw.ellipse((x-6,y-6,x+6,y+6),fill="#c43c39",outline="white")
        draw.text((x+7,y-13),str(i+1),fill="#222222",font=ImageFont.truetype(FONT,13))
    draw.text((35,20),f"{title}; длина {result.length:.2f}",fill="#111111",font=font)
    canvas.save(path)



def draw_convergence(results, path: Path):
    canvas=Image.new("RGB",(1100,600),"white"); draw=ImageDraw.Draw(canvas)
    box=(125,75,1030,490); draw.rectangle(box,outline="#333333",width=2)
    colors={"path":"#c43c39","cycle":"#2878b5"}
    values=[row["best_length"] for result in results.values() for row in result.history]
    low=min(values); high=max(values)
    small=ImageFont.truetype(FONT,15)
    for tick in range(5):
        fraction=tick/4
        y=box[1]+(box[3]-box[1])*fraction
        value=high-(high-low)*fraction
        draw.line((box[0],y,box[2],y),fill="#dddddd",width=1)
        draw.text((24,y-9),f"{value:.0f}",fill="#333333",font=small)
        x=box[0]+(box[2]-box[0])*fraction
        draw.line((x,box[3],x,box[3]+6),fill="#333333",width=1)
        max_iterations=max(len(result.history) for result in results.values())
        iteration=1+round((max_iterations-1)*fraction)
        draw.text((x-15,box[3]+10),str(iteration),fill="#333333",font=small)
    for label,result in results.items():
        points=[]
        for row in result.history:
            x=box[0]+(box[2]-box[0])*(row["iteration"]-1)/max(1,len(result.history)-1)
            y=box[3]-(box[3]-box[1])*(row["best_length"]-low)/max(high-low,1e-9)
            points.append((x,y))
        draw.line(points,fill=colors[label],width=3)
        legend={"path":"путь","cycle":"цикл TSP"}[label]

draw.text((850,20+25*list(results).index(label)),legend,fill=colors[label],font=ImageFont.truetype(FONT,18
))
    draw.text((125,20),"Сходимость муравьиной колонии",fill="#111111",font=ImageFont.truetype(FONT,22))
    draw.text((480,545),"Итерация",fill="#333333",font=ImageFont.truetype(FONT,17))
    draw.text((24,48),"Длина маршрута",fill="#333333",font=small)
    canvas.save(path)



def main(verbose: bool = False):
    distances=distance_matrix()
    reference_route=[city-1 for city in OPTIMAL_TOUR_1BASED]
    reference_cycle=route_length(reference_route,distances,closed=True)
    reference_open=route_length(reference_route,distances,closed=False)
    base={}
    for label,closed,seed in (("path",False,44),("cycle",True,45)):

base[label]=AntColony(distances,ants=40,iterations=200,alpha=1.0,beta=3.0,evaporation=.35,seed=seed).run(c
losed=closed)
        if not validate_route(base[label].route,len(COORDINATES)):
            raise RuntimeError(f"invalid route generated in {label} run")
    draw_route(base["path"],RESULTS/"bayg29_hamiltonian_path.png","Bayg29 — гамильтонов путь (без
возврата)")
    draw_route(base["cycle"],RESULTS/"bayg29_tsp_cycle.png","Bayg29 — цикл TSP (с возвратом)")
    draw_convergence(base,RESULTS/"convergence.png")
    study=[]
    for ants,evaporation,seed in ((20,.25,51),(40,.25,52),(20,.45,53),(40,.45,54)):

result=AntColony(distances,ants=ants,iterations=120,alpha=1.0,beta=3.0,evaporation=evaporation,seed=seed).
run(closed=True)

study.append({"ants":ants,"evaporation":evaporation,"best_length":result.length,"generations":len(result.h
istory),"time_seconds":result.elapsed_seconds,"route":[x+1 for x in result.route]})
    with (RESULTS/"parameter_study.csv").open("w",encoding="utf-8",newline="") as stream:
```

```
writer=csv.DictWriter(stream,fieldnames=["ants","evaporation","best_length","generations","time_seconds","
route"]);writer.writeheader()
        for row in study: writer.writerow({**row,"route":" ".join(map(str,row["route"]))})

summary={"variant":4,"dataset":"Bayg29","coordinates":COORDINATES,"optimal_tour_1based":OPTIMAL_TOUR_1BASE
D,
             "reference_cycle_length":reference_cycle,"reference_open_path_length":reference_open,
             "path_reference_note":"Removing the closing edge from the known optimal cycle gives a
feasible path upper bound, not a proven open-path optimum.",
             "parameters":{"ants":40,"iterations":200,"alpha":1.0,"beta":3.0,"evaporation":.35,"q":100.0},
             "base":{label:{"route_1based":[x+1                  for                  x                  in
result.route],"length":result.length,"closed":result.closed,"elapsed_seconds":result.elapsed_seconds,"hist
ory":result.history} for label,result in base.items()},"study":study}

(RESULTS/"results_summary.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding="utf-
8")
    print("Лабораторная работа 6 — муравьиная колония; вариант 4, Bayg29 (29 городов)")
    print("Расстояния рассчитаны по координатам из методички; формируется полный маршрут по всем
городам.")
    print(f"Параметры базовой серии: 40 муравьев, 200 итераций, alpha=1, beta=3, испарение=0.35; режим:
{'подробный' if verbose else 'краткий'}.")
    print("\nЧасть 1 — кратчайший гамильтонов путь, без ребра возврата:")
    print("  "+format_run("Найденный путь",base["path"],reference_open))
    print("   Сравнение только с путем, полученным удалением замыкающего ребра известного оптимального
тура; это верхняя оценка, не доказанный оптимум пути.")
    print("\nЧасть 2 — задача коммивояжера, замкнутый цикл:")
    print("  "+format_run("Найденный тур",base["cycle"],reference_cycle))
    print("\nКонтрольные итерации лучшего результата:")
    for label,result in base.items():
        print(f"  {label}:")
        rows=result.history if verbose else selected_history(result.history)
        for row in rows:
            print(f"     итерация {row['iteration']:>3}: лучший маршрут={row['best_length']:.3f}; средняя
длина={row['mean_length']:.3f}")
    print("\nПараметрическое исследование TSP:")
    print(f"  {'муравьи':>8} {'испарение':>10} {'длина тура':>12} {'итерации':>10} {'время, с':>10}")
    for row in study: print(f"  {row['ants']:>8} {row['evaporation']:>10.2f} {row['best_length']:>12.3f}
{row['generations']:>10} {row['time_seconds']:>10.3f}")
    print("\nФайлы результатов:")
    for                                    filename                                    in
("results_summary.json","parameter_study.csv","bayg29_hamiltonian_path.png","bayg29_tsp_cycle.png","conver
gence.png"):
        print(f"  {RESULTS/filename}")



if __name__=="__main__":
    parser=argparse.ArgumentParser(description="Муравьиный алгоритм для пути и TSP Bayg29")
    parser.add_argument("--verbose",action="store_true",help="печатать каждую итерацию")
    main(parser.parse_args().verbose)
```